

Mocking Dependencies in Spring Tests using Mockito

Exercise 1: Mocking a Service Dependency in a Controller Test

Problem

Test a Spring controller by mocking the service layer.

Setup

Use `@WebMvcTest`, `@MockBean` and `MockMvc`.

Java Implementation

```
@WebMvcTest(UserController.class)
class UserControllerTest{
    @Autowired MockMvc mvc;
    @MockBean UserService service;

    @Test
    void testGetUser() throws Exception{
        User u=new User();
        u.setId(1L);
        u.setName("John");
        when(service.getUserById(1L)).thenReturn(u);

        mvc.perform(get("/users/1"))
            .andExpect(status().isOk());
    }
}
```

Output

Program executes successfully and the test passes.

Exercise 2: Mocking a Repository in a Service Test

Problem

Test `UserService` by mocking `UserRepository`.

Setup

Use `@Mock` and `@InjectMocks`.

Java Implementation

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest{
    @Mock UserRepository repo;
```

```
@InjectMocks UserService service;
```

```
@Test
void testGetUser(){
    User u=new User();
    u.setId(1L);
    when(repo.findById(1L)).thenReturn(Optional.of(u));
    assertEquals(1L,service.getUserById(1L).getId());
}
}
```

Output

Program executes successfully and the test passes.

Exercise 3: Mocking a Service Dependency in an Integration Test

Problem

Integration test with mocked service dependency.

Setup

Use `@SpringBootTest`, `@AutoConfigureMockMvc` and `@MockBean`.

Java Implementation

```
@SpringBootTest
@AutoConfigureMockMvc
class UserIntegrationTest{

    @Autowired MockMvc mvc;
    @MockBean UserService service;

    @Test
    void testUser() throws Exception{
        User u=new User();
        u.setId(1L);
        u.setName("John");
        when(service.getUserById(1L)).thenReturn(u);

        mvc.perform(get("/users/1"))
            .andExpect(status().isOk());
    }
}
```

Output

Program executes successfully and the test passes.